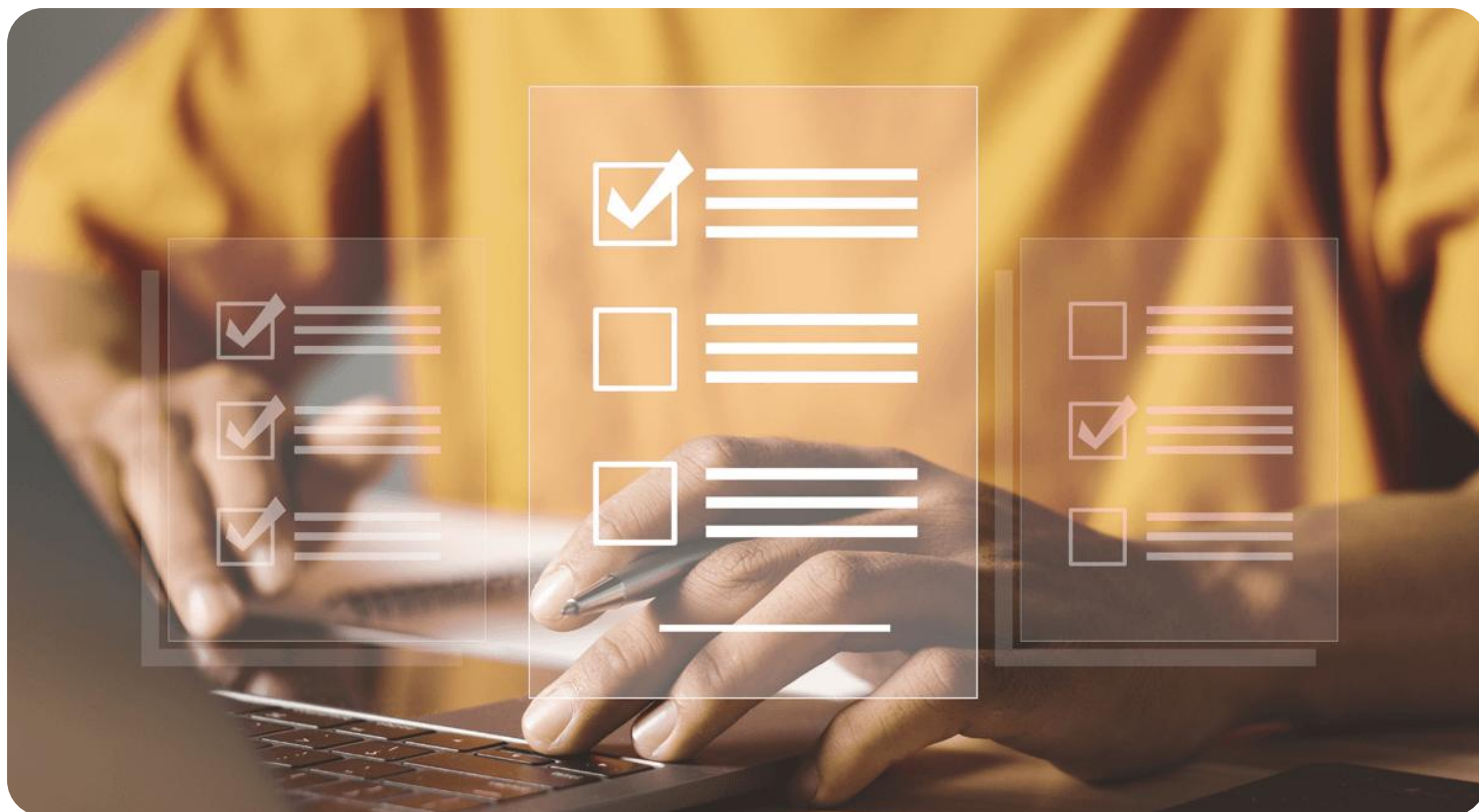




Semana 7

Desarrollo Backend (PBY2202)

Formato de respuesta

Nombre estudiante: Francisco Henriquez – Raúl Zúñiga	
Carrera: Desarrollo de Aplicaciones	
Profesor: Jorge Carmona	Fecha: 06-07-2026

Descripción de la actividad

En esta séptima semana realizarás un tipo de actividad formativa grupal llamada “Documentando Microservicios con OpenAPI”. Esta consiste en la elaboración de la documentación de la arquitectura y los endpoints de los microservicios utilizando OpenAPI Specification (OAS), asegurando la consistencia y alineación con los estándares establecidos.

Instrucciones específicas

Para el desarrollo de la actividad formativa de la semana, recordemos brevemente los requerimientos del caso:

Contexto

El sistema backend de Minimarket Plus que vienes desarrollando, implementa microservicios para gestionar productos, carritos, inventario, ventas y usuarios con diferentes roles (cliente, cajero, administrador). Ahora, necesitas generar la documentación técnica de estos servicios utilizando OpenAPI, para garantizar que otros desarrolladores y equipos puedan comprender, integrar y mantener fácilmente el sistema, fortaleciendo la escalabilidad y el control de las operaciones del minimarket.

Para esta semana deberás:

- Documentar los endpoints REST existentes utilizando **anotaciones OpenAPI (springdoc-openapi)**.
- Generar la interfaz interactiva con **Swagger UI** para verificar la documentación.
- Asegurar que los endpoints cuenten con ejemplos claros, definiciones de parámetros y respuestas.
- Validar la consistencia exportando el JSON generado y probándolo en herramientas externas como Postman.



Importante

El código base del proyecto "MINIMARKET PLUS" será proporcionado para su descarga a través del Aula Virtual. Deberás utilizar este backend como base para la actividad, enfocándote en la configuración del entorno de pruebas unitarias y el diseño de las pruebas necesarias para garantizar la calidad del desarrollo, según los requerimientos establecidos.

Ahora que has revisado y analizado el caso, tendrás que realizar los siguientes pasos:

Paso 1: Configuración del entorno de pruebas

- Asegúrate de que el proyecto esté configurado correctamente en tu entorno de desarrollo (IDE).
- Verifica que el archivo pom.xml contenga la dependencia de springdoc-openapi-starter-webmvc-ui.
- Sincroniza el proyecto para asegurarte de que las dependencias se instalen correctamente.

Paso 2: Documentación de la entidad Producto

1. Definición de endpoints:
 - Revisa el controlador ProductoController incluido en el proyecto.
 - Identifica los endpoints disponibles. Por ejemplo:
 - GET /api/productos: obtiene la lista de productos.
 - POST /api/productos: crea un nuevo producto.
 - Usa anotaciones de OpenAPI como @Operation y @ApiResponse para documentar los métodos.

2. Validación de la documentación:

- Ejecuta la aplicación y accede a Swagger UI en <http://localhost:8080/swagger-ui.html>.
- Confirma en Swagger UI que los endpoints estén documentados correctamente y que la información sea clara y completa.

Paso 3: Documentación de la entidad Carrito

1. Definición de endpoints:

- Identifica los métodos en el controlador CarritoController. Por ejemplo:
 - POST /api/carritos: añade un producto al carrito.
 - DELETE /api/carritos/{id}: elimina un producto del carrito.
- Documenta los endpoints utilizando OpenAPI.

2. Incluir ejemplos de parámetros y respuestas:

- Utiliza la sección components en OpenAPI para definir modelos reutilizables.
- Prueba en Swagger UI para confirmar que los modelos y respuestas estén bien definidos.

Paso 4: Pruebas de validación y consistencia

1. Validar endpoints en Swagger UI:

- Asegúrate de que todos los métodos documentados en OpenAPI se comporten correctamente al ser probados desde <http://localhost:8080/swagger-ui.html>.

2. Prueba de consistencia externa:

- Genera el archivo JSON desde <http://localhost:8080/v3/api-docs>.
- Importa este archivo en Postman y realiza pruebas para validar que los endpoints respondan según lo documentado.

Paso 5: Documentación técnica y análisis de resultados

Redacta un informe directamente en este documento, que incluya:

1. Resumen técnico del avance respecto a semanas anteriores

Explica cómo el trabajo en autenticación, pruebas y microservicios realizado en las semanas 4 a 6 se integra ahora con la documentación formal de los endpoints.

2. Análisis de la documentación generada

Detalla:

- ¿Qué endpoints fueron documentados y con qué nivel de detalle?
- ¿Qué problemas encontraste al documentar o validar los contratos de los servicios?
- ¿Qué mejoras aplicaste a la estructura de datos o a los controladores para clarificar la documentación?

3. Evidencia de ejecución

Incluye:

- Capturas de Swagger UI mostrando los endpoints documentados.
- JSON exportado desde /v3/api-docs.
- Importación del JSON en Postman u otra herramienta, validando la consistencia.

4. Reflexión técnica

- ¿Cómo aporta OpenAPI a la calidad y mantenibilidad del backend?
- ¿Qué estrategias usarás para mantener la documentación actualizada en futuras versiones?
- ¿Cómo mejorarías la experiencia del usuario al interactuar con la documentación generada?

Paso 6: Subida de código a GitHub

- Sube el código actualizado con las anotaciones OpenAPI y tu archivo exportado /v3/api-docs.
- Incluye en el README.md instrucciones básicas para iniciar el proyecto y visualizar Swagger UI.

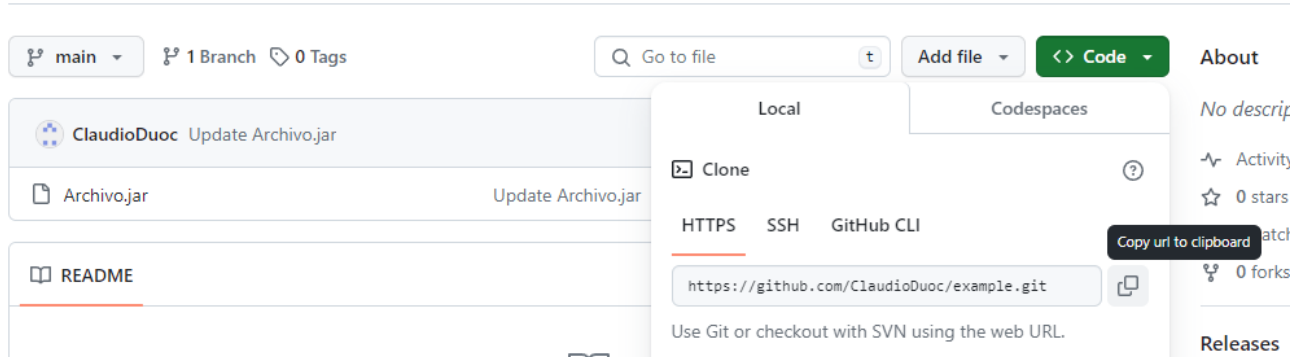
Si no has creado tu cuenta aún, puedes hacerlo a través del siguiente enlace:

<https://github.com/>

Posteriormente, desde el repositorio, deberás generar un enlace de tu proyecto:

Figura 1

Enlace de proyecto GitHub



Nota. Ejemplo genérico de donde se extrae un enlace en GitHub. GitHub (s.f.). *GitHub*.

<https://github.com/>

Paso 7: Para tu entrega, no olvides adjuntar este documento con tus entregables y subir al AVA, junto con el enlace de GitHub a adjuntar en la sección “Entrega”.

Entregables

En este apartado deberás adjuntar lo solicitado en los pasos anteriores sobre:

1. Resumen técnico del avance respecto a semanas anteriores

(Describe brevemente cómo el trabajo previo permitió implementar esta documentación y qué relación tiene con el proyecto completo.)

Durante las semanas anteriores se trabajó en la base funcional del backend de autenticación, estructura de servicios, persistencia con JPA, separación por capas y control de acceso. Ese avance permitió llegar a esta semana con controladores y entidades ya definidos, por lo que la documentación OpenAPI pueda contruirse sobre contratos REST reales y no sobre un diseño preliminar.

La integración actual busca fortalecer el proyecto porque transforma los endpoints existentes en una interfaz consumible por otros equipos (para casos reales). En particular, la implementación de Swagger UI facilitará la verificación visual de los servicios, mientras que el JSON generado por `/V3/api-docs` permitirá validar externa e interoperabilidad con herramientas similares como las que hemos usado a lo largo de la carrera como es POSTMAN

Análisis de la documentación generada con OpenAPI

(Explica qué endpoints fueron documentados, problemas encontrados y mejoras aplicadas.)

Los Endpoints documentados

En el presente proyecto se documentaron los siguientes controladores:

- Producto
- Carrito

En **Producto** se documentaron los siguientes endpoints:

- GET /api/productos
- GET /api/productos/{id}
- POST /api/productos
- PUT /api/productos/{id}
- DELETE /api/productos/{id}

En **Carrito** se documentaron los siguientes endpoints:

- GET /api/carrito
- GET /api/carrito/{id}
- POST /api/carrito
- PUT /api/carrito/{id}
- DELETE /api/carrito/{id}

Problemas encontrados durante la implementación

El principal problema que fue encontrado durante el proyecto fue que no tenía la dependencia de springdoc-openapi-starter-webmvc-ui, por lo que Swagger UI no estaba disponible inicialmente. Además, la configuración de seguridad bloqueaba el acceso a /swagger-ui.html y /v3/api-docs.

Otro punto que se consideró fue que las entidades usaban relaciones @ManyToOne, lo que nos generó documentación más completa, ya que se tenían que describir adecuadamente para entender los modelos relacionales.

Mejoras aplicadas

- 1- Se agregaron las dependencias de Springdoc en el archivo pom.xml.
- 2- Se habilitó el acceso público a Swagger UI y al JSON de OpenAPI en la configuración de seguridad.
- 3- Se documentaron las entidades con @Schema para hacer más claro el modelo de los datos.
- 4- Se mantuvieron las estructuras originales de los controladores. Solo se agregó el metadato de la documentación.

Evidencia de ejecución y validación

(Adjunta capturas de Swagger UI, el JSON exportado y muestra la importación en Postman o herramienta similar.)

Evidencia de la ejecución en Swagger UI

Para evidenciar la implementación, se ejecutó la aplicación desde nuestro IDE y una vez que la aplicación se encontraba corriendo, nos dirigimos a nuestro navegador de confianza y escribimos la siguiente URL <http://localhost:8080/swagger-ui.html> Lo anterior nos abrirá la página de swagger y nos mostrará lo siguiente:

The screenshot shows the Swagger UI interface for the Minimarket Plus API. The browser address bar displays `localhost:8080/swagger-ui/index.html`. The Swagger logo is in the top left, and a search bar with `/v3/api-docs` and an `Explore` button is in the top right. The main heading is **Minimarket Plus API** with version `1.0` and `OAS 3.0` tags. Below this, it says `/v3/api-docs`, `Documentación de los servicios REST del sistema Minimarket Plus`, and `Contact Equipo de desarrollo`.

A `Servers` dropdown menu shows `http://localhost:8080 - Generated server url`.

The **usuario-controller** section is expanded, showing the following endpoints:

- `GET /api/usuarios/{id}`
- `PUT /api/usuarios/{id}`
- `DELETE /api/usuarios/{id}`
- `GET /api/usuarios`
- `POST /api/usuarios`

The **producto-controller** section is also expanded, showing the following endpoints:

- `GET /api/productos/{id}` Buscar producto por ID
- `PUT /api/productos/{id}` Actualizar producto
- `DELETE /api/productos/{id}` Eliminar producto
- `GET /api/productos` Listar productos
- `POST /api/productos` Crear producto

inventario-controller

GET	/api/inventario/{id}	▼
PUT	/api/inventario/{id}	▼
DELETE	/api/inventario/{id}	▼
GET	/api/inventario	▼
POST	/api/inventario	▼

detalle-venta-controller

GET	/api/detalle-ventas/{id}	▼
PUT	/api/detalle-ventas/{id}	▼
DELETE	/api/detalle-ventas/{id}	▼
GET	/api/detalle-ventas	▼
POST	/api/detalle-ventas	▼

categoria-controller

GET	/api/categorias/{id}	▼
PUT	/api/categorias/{id}	▼
DELETE	/api/categorias/{id}	▼
GET	/api/categorias	▼
POST	/api/categorias	▼

carrito-controller

GET	/api/carrito/{id} Buscar item del carrito por ID	▼
PUT	/api/carrito/{id} Actualizar carrito	▼
DELETE	/api/carrito/{id} Eliminar item del carrito	▼
GET	/api/carrito Listar carrito	▼
POST	/api/carrito Agregar producto al carrito	▼

venta-controller

GET	/api/ventas	▼
POST	/api/ventas	▼
GET	/api/ventas/{id}	▼

hola-mundo-controller

GET	/public/hola	▼
-----	--------------	---

Además, la herramienta nos entregó los Schemas de nuestra aplicación.




```

Producto ▾ {
  description:      Producto disponible en el minimarket
  id                ▾ integer($int64)
                   example: 1
                   Identificador del producto
  nombre            ▾ string
                   example: Leche Entera
                   Nombre del producto
  precio            ▾ number($double)
                   example: 1290
                   Precio del producto
  stock             ▾ integer($int32)
                   example: 25
                   Stock disponible
  categoria          {
    description:      Categoría asociada al producto
    id                > [...]
    nombre            > [...]
    productos         > [...]
  }
}

```

```

Inventario ▾ {
  id                ▾ integer($int64)
  producto          Producto > {...}
  cantidad          ▾ integer($int32)
  tipoMovimiento    ▾ string
  fechaMovimiento   ▾ string($date-time)
}

```

```

DetalleVenta ▾ {
  id                ▾ integer($int64)
  venta             Venta ▾ {
    id              > [...]
    usuario         Usuario > {...}
    fecha           > [...]
    detalles        > [...]
  }
  producto          Producto > {...}
  cantidad          ▾ integer($int32)
  precio            ▾ number($double)
}

```

```

Venta ▾ {
  id                ▾ integer($int64)
  usuario           Usuario > {...}
  fecha             ▾ string($date-time)
  detalles          ▾ [ ▾ {
    id              > [...]
    venta           Venta > {...}
    producto        Producto > {...}
    cantidad        > [...]
    precio          > [...]
  }]
}

```

```

Carrito ▾ {
  description:      Item agregado al carrito de compra
  id                ▾ integer($int64)
                   example: 1
                   Identificador del item del carrito
  usuario           Usuario ▾ {
    id              ▾ integer($int64)
    username        ▾ string
    password        ▾ string
    roles           ▾ [
                       uniqueItems: true
                     ]
    {
      id            ▾ integer($int64)
      nombre        ▾ string
      usuarios      ▾ [

```

Imagen de Swagger con todos los endpoint expuesto inicialmente.

 **Swagger**
Supports by SMARTBEAR

/v3/api-docs

Explore

Minimarket Plus API

1.0OAS 3.0

[/v3/api-docs](#)

Documentación de los servicios REST del sistema Minimarket Plus

[Contact Equipo de desarrollo](#)

Servers

http://localhost:8080 - Generated server url

usuario-controller

producto-controller

inventario-controller

detalle-venta-controller

categoria-controller

carrito-controller

venta-controller

hola-mundo-controller

Schemas

Rol >

Usuario >

Categoria >

Producto >

Inventario >

DetalleVenta >

Venta >

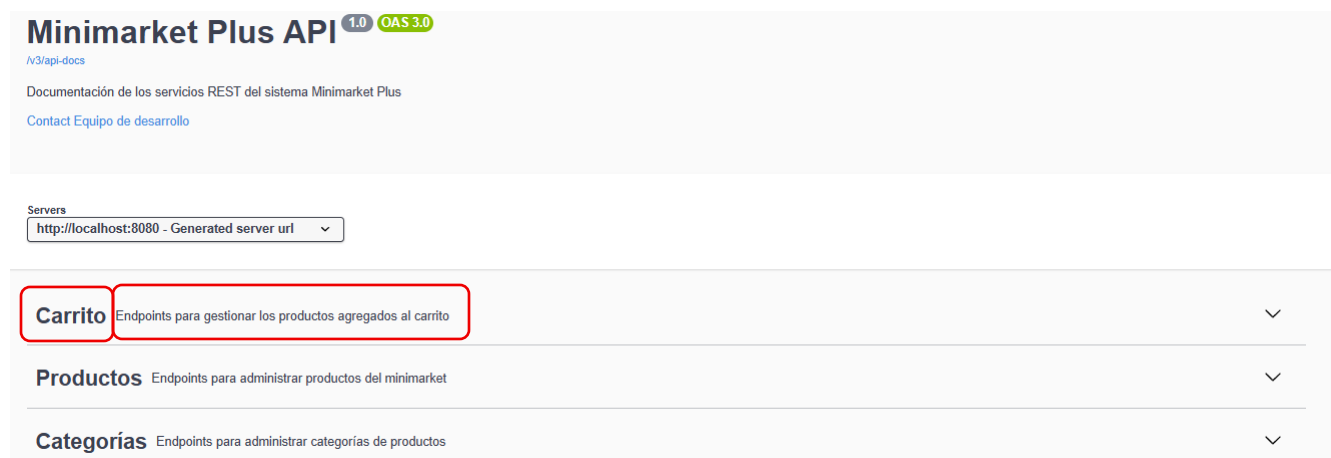
Carrito >

Un punto importante de la utilización de Swagger es que podemos explicar que significa cada Endpoint mediante la implementación de la notación `@tag` seguido de un nombre y una descripción. Ejemplo: `@tag(name = "Nombre del endpoint", descripción="Descripción de para que sirve el endpoint")`

Imagen de la explicación anterior para el endpoint carrito

```
@RestController
@RequestMapping("/api/carrito")
@Tag(name = "Carrito", description = "Endpoints para gestionar los productos agregados al carrito")
public class CarritoController {
```

Al implementar los tag, nuestro Swagger se modifica y si recargamos la aplicación y volvemos a refrescar la página podremos ver que nos muestra los endpoints de la siguiente forma.



También se agregaron descripciones a cada endpoints en específico, mediante la anotación `@Operation` que al igual que en el caso de la anotación `@tag` se entrega un nombre y su descripción. También como buena practica y como el profesor indicó es necesario agregar un `@ApiResponse`, esto es importante ya que me va a indicar lo que debe responder el endpoint frente a diferentes situaciones.

Imagen del código con la implementación del `@Operation` y `@ApiResponse`

```
@GetMapping("/{id}")
@Operation(summary = "Buscar item del carrito por ID", description = "Retorna un item específico del carrito")
@ApiResponse({
    @ApiResponse(responseCode = "200", description = "Item encontrado"),
    @ApiResponse(responseCode = "404", description = "Item no encontrado")
})
```

Imagen de Swagger como se demuestra la implementación

GET

/api/carrito/{id} Buscar ítem del carrito por ID

^

Retorna un ítem específico del carrito

Parameters

Try it out

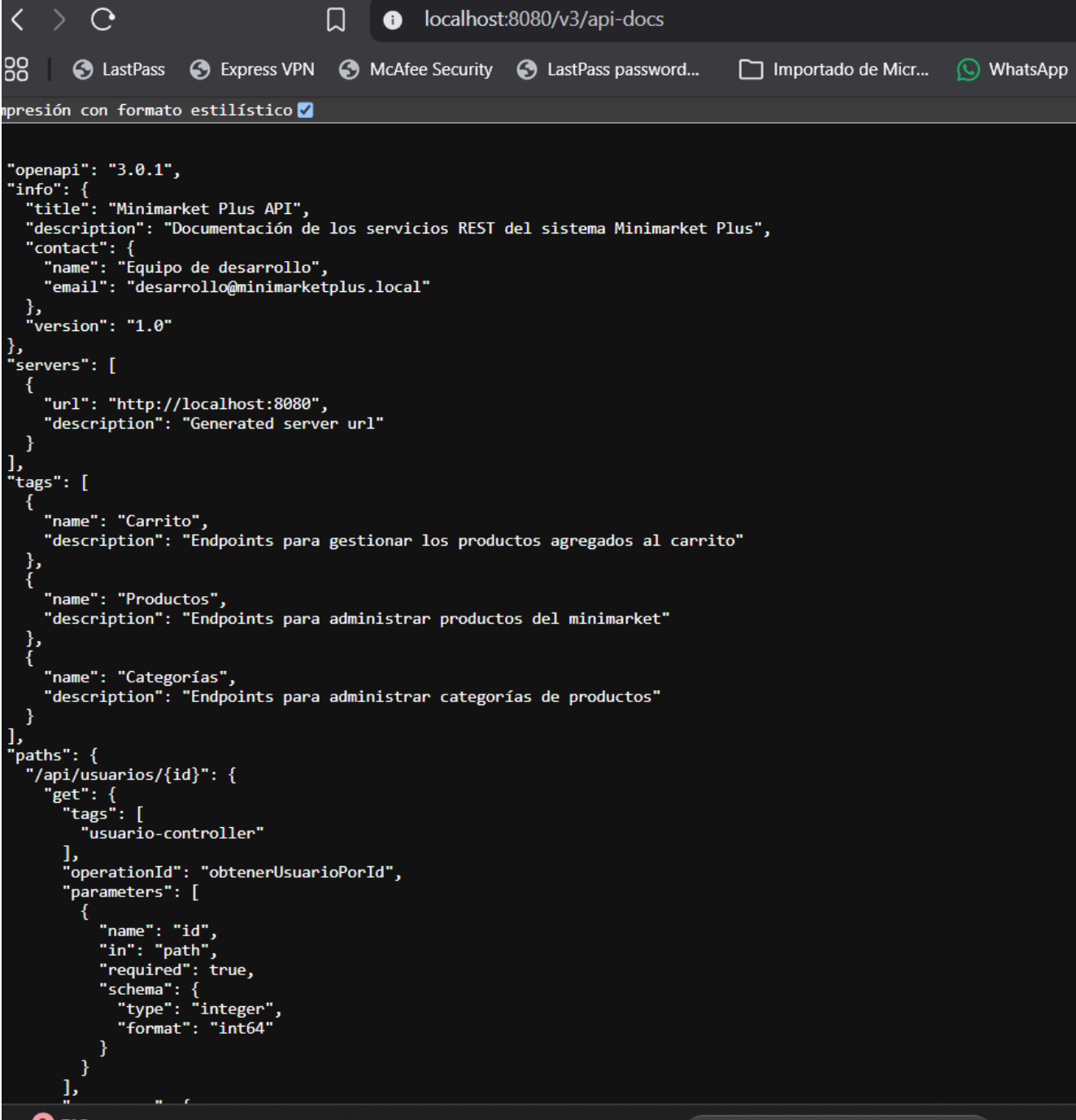
Name	Description
id * required integer(\$int64) (path)	id

Responses

Code	Description	Links
200	Item encontrado	No links
Media type */*		
Controls Accept header.		
Example Value Schema		
<pre>{ "id": 1, "usuario": { "id": 0, "username": "string", "password": "string", "roles": [{ "id": 0, "nombre": "string", "usuarios": ["string"] }] }, "producto": { "id": 1, "nombre": "Leche Entera", "precio": 1290, "stock": 25, "categoria": { "id": 0, "nombre": "string", "productos": ["string"] } } }</pre>		
404	Item no encontrado	No links
Media type */*		
Example Value Schema		
<pre>{ "id": 1, "usuario": { "id": 0, "username": "string", "password": "string", "roles": [{ "id": 0, "nombre": "string", "usuarios": ["string"] }] }, "producto": { "id": 1, "nombre": "Leche Entera", "precio": 1290, "stock": 25, "categoria": { "id": 0, "nombre": "string", "productos": ["string"] } } }</pre>		

Exportación del código a JSON

También podemos exportar todo el código que generamos en JSON en caso de que lo soliciten. Para ello debemos ingresar a la siguiente URL desde nuestro navegador de confianza <http://localhost:8080/v3/api-docs> esto nos devuelve todo el código con nuestras anotaciones en JSON



```

"openapi": "3.0.1",
"info": {
  "title": "Minimarket Plus API",
  "description": "Documentación de los servicios REST del sistema Minimarket Plus",
  "contact": {
    "name": "Equipo de desarrollo",
    "email": "desarrollo@minimarketplus.local"
  },
  "version": "1.0"
},
"servers": [
  {
    "url": "http://localhost:8080",
    "description": "Generated server url"
  }
],
"tags": [
  {
    "name": "Carrito",
    "description": "Endpoints para gestionar los productos agregados al carrito"
  },
  {
    "name": "Productos",
    "description": "Endpoints para administrar productos del minimarket"
  },
  {
    "name": "Categorías",
    "description": "Endpoints para administrar categorías de productos"
  }
],
"paths": {
  "/api/usuarios/{id}": {
    "get": {
      "tags": [
        "usuario-controller"
      ],
      "operationId": "obtenerUsuarioPorId",
      "parameters": [
        {
          "name": "id",
          "in": "path",
          "required": true,
          "schema": {
            "type": "integer",
            "format": "int64"
          }
        }
      ]
    }
  }
},

```

Reflexión técnica sobre la calidad y mantenibilidad

(Redacta cómo OpenAPI contribuye a la calidad del backend y qué estrategias usarás para mantener actualizada la documentación.)

Como reflexión tenemos que OpenAPI aporta calidad al backend porque formaliza los contratos de los servicios y reduce la ambigüedad al integrarlos. También mejora el mantenimiento, ya que los cambios en los endpoints quedan visibles en Swagger UI y pueden validarse en forma rápida.

Para tener documentación actualizada en futuras versiones recomendamos lo siguiente:

- Documentar los controladores al mismo tiempo que se implementan.
- Revisar Swagger UI en cada entrega
- Validar el JSON generando como parte de las pruebas de integración.
- Mantener ejemplos de entrada y salida alineados con modelos reales.

La experiencia del usuario podría ser mejor si consideramos lo siguiente:

- Más ejemplos de request body
- Descripciones más detalladas de validaciones y errores
- Agrupación por tags o módulos funcionales
- Inclusión de autenticación JWT en Swagger cuando corresponda.

2. Recomendaciones de mejora para futuras versiones del proyecto

(Incluye sugerencias para optimizar la documentación o la estructura del backend.)

Si bien es cierto que la aplicación la estamos trabajando desde los inicios del semestre, en esta oportunidad trabajamos la aplicación desde cero, mediante el link que nos entregó DUOC, si consideramos que estamos trabajando este archivo como si fuera el primero y sin las implementaciones que hemos hecho a lo largo de toda la experiencia, creemos que sería bueno implementar las siguientes mejoras futuras:

- Crear DTOs para evitar exponer entidades directamente.
- Agregar ejemplos específicos en request7response de todos los endpoints.
- Documentar también los módulos de Usuario, Venta, Inventario y DetalleVenta.
- Incorporar seguridad en OpenAPI si se desea probar autenticación desde Swagger
- Automatizar la exportación del JSON de OpenAPI en el pipeline de construcción.



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.